计算机图形学 - 补充资料

Computer Graphics

@何晓曦

2020版

第二章 渲染管线发展史

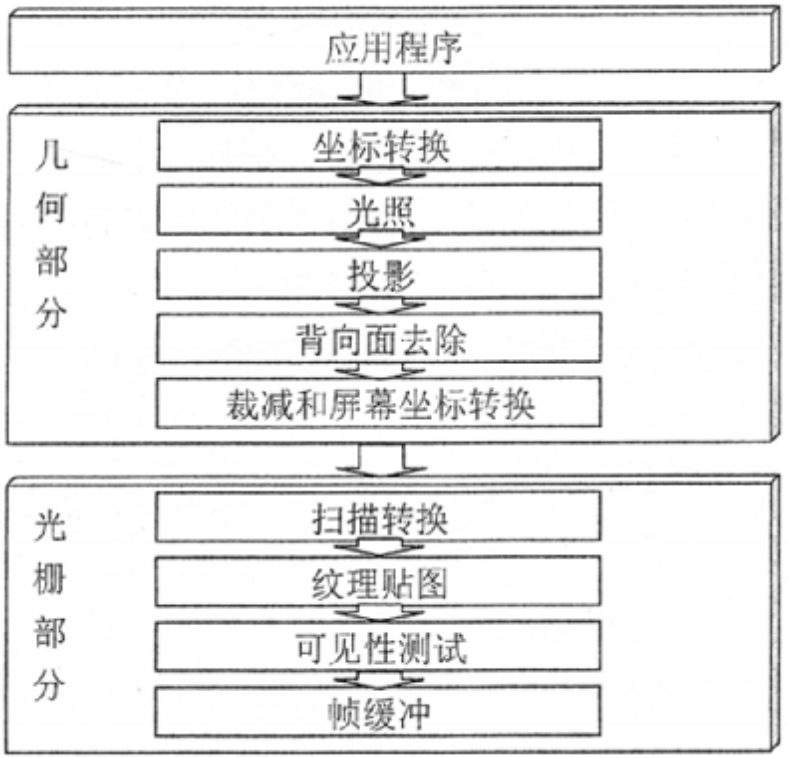
⊕ 渲染管线发展史

- 渲染管线做的事情就是让计算机完成图形功能，而图形功能其实就是做了下面图里这件事情。

有了光源、房子和相机，然后计算相机上面拍出的照片到底是什么样子的。

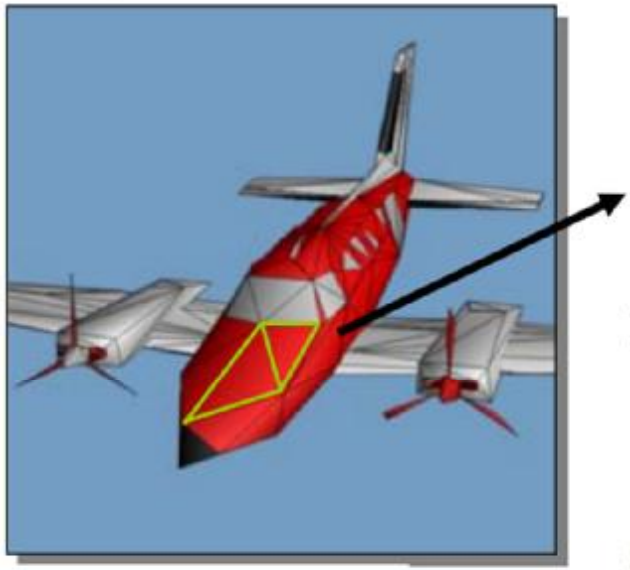
⊕ 渲染管线发展史

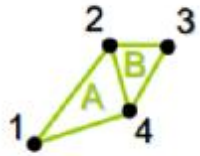
- 这个任务比较复杂，然后大家就把它分成了几个步骤来完成。



⊕ 渲染管线发展史

- 这里面几何部分处理的都是顶点，而光栅部分处理的都是像素，比如说下面这个飞机：





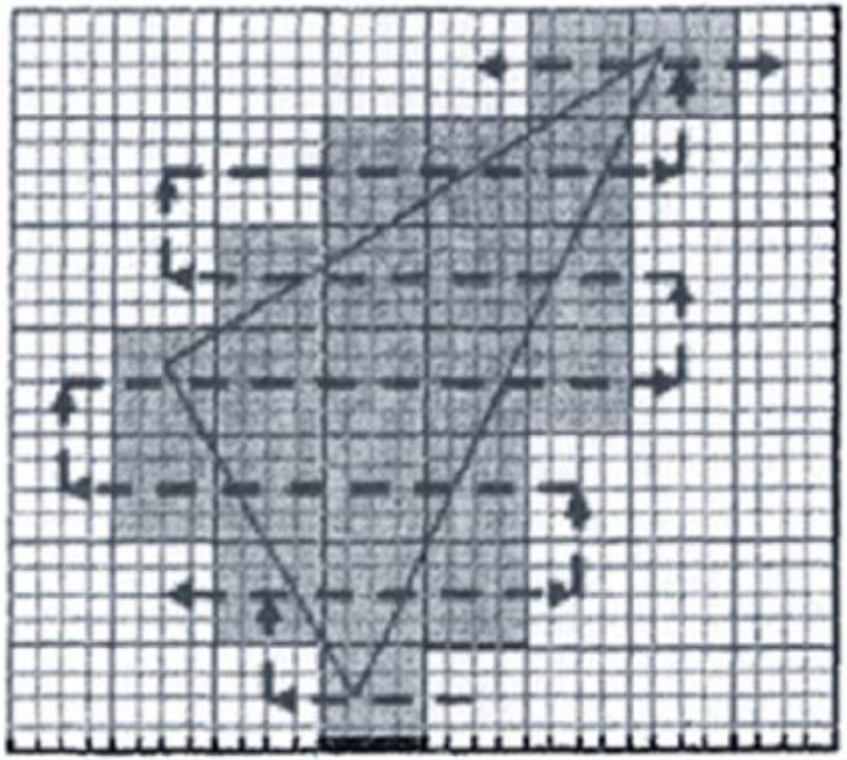
Vertex Array
(x1, y1, z1)
(x2, y2, z2)
…

Index Array
V1, V2, V4 – represents Triangle A
V4, V2, V3 – represents Triangle B

Vertices are only stored once.
Triangles point to their vertices.

⊕ 渲染管线发展史

- 几何部分的时候，算来算去处理的都是1,2,3,4这样几个顶点的信息，而到了后面光栅部分，经过光栅化就成了这样

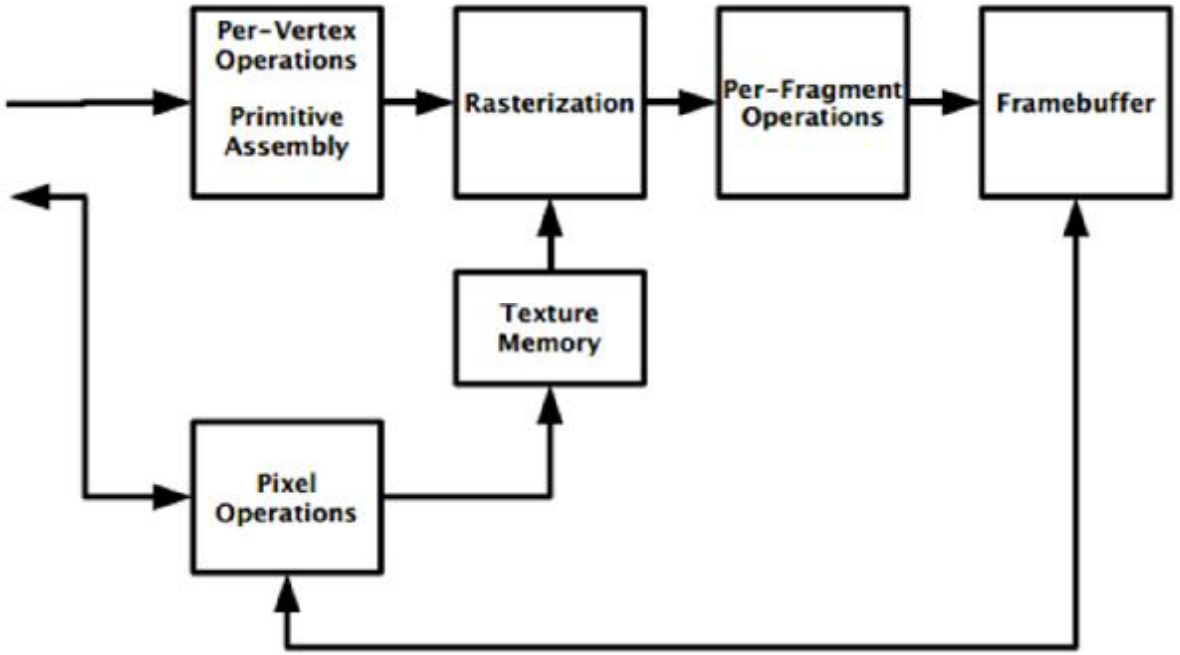


⊕ 渲染管线发展史

- 这也就是最早的固定管线，上面的每一个步骤（比如光照、贴图等）里面都会有一些参数（比如光照模式，纹理模式等等）；
- 整个渲染的过程就好像是在流水线上的每一道工序上面都有几个可选的开关，然后你要想实现不同的渲染效果，那就去花式扳开关就好了。

⊕ 渲染管线发展史

- 比如这个是固定管线的OpenGL ES 1.1的样子



⊕ 渲染管线发展史

- 这个时候的API比较典型的都是这样的

```
GLAPI void APIENTRY glLightModelf (GLenum pname, GLfloat param);
GLAPI void APIENTRY glLightModelfv (GLenum pname, const GLfloat *params);
GLAPI void APIENTRY glLightModelx (GLenum pname, GLfixed param);
GLAPI void APIENTRY glLightModelxv (GLenum pname, const GLfixed *params);
GLAPI void APIENTRY glLightf (GLenum light, GLenum pname, GLfloat param);
GLAPI void APIENTRY glLightfv (GLenum light, GLenum pname, const GLfloat *params);
GLAPI void APIENTRY glLightx (GLenum light, GLenum pname, GLfixed param);
GLAPI void APIENTRY glLightxv (GLenum light, GLenum pname, const GLfixed *params);
GLAPI void APIENTRY glLineWidth (GLfloat width);
GLAPI void APIENTRY glLineWidthx (GLfixed width);
GLAPI void APIENTRY glLoadIdentity (void);
GLAPI void APIENTRY glLoadMatrixf (const GLfloat *m);
GLAPI void APIENTRY glLoadMatrixx (const GLfixed *m);
GLAPI void APIENTRY glLogicOp (GLenum opcode);
GLAPI void APIENTRY glMaterialf (GLenum face, GLenum pname, GLfloat param);
GLAPI void APIENTRY glMaterialfv (GLenum face, GLenum pname, const GLfloat *params);
GLAPI void APIENTRY glMaterialx (GLenum face, GLenum pname, GLfixed param);
GLAPI void APIENTRY glMaterialxv (GLenum face, GLenum pname, const GLfixed *params);
GLAPI void APIENTRY glMatrixMode (GLenum mode);
GLAPI void APIENTRY glMultMatrixf (const GLfloat *m);
GLAPI void APIENTRY glMultMatrixx (const GLfixed *m);
```

⊕ 渲染管线发展史

- 可以看出来，这就是让大家花式扳开关的节奏
  但是大家的欲望是无限的，这么有限的扳开关的模式，渲染的效果实在是不理想;
- 后来大家就把模式换成了流水线上坐着几个工人（Shader），然后给工人下命令（可编程）的方式了，这样的话只要工人忙得过来，那我就可以自己定义各种光照模型，贴图方式了，程序员的可发挥空间就大得多了。

⊕ 渲染管线发展史

- 比如到了OpenGL ES 2.0以后就变成了这样

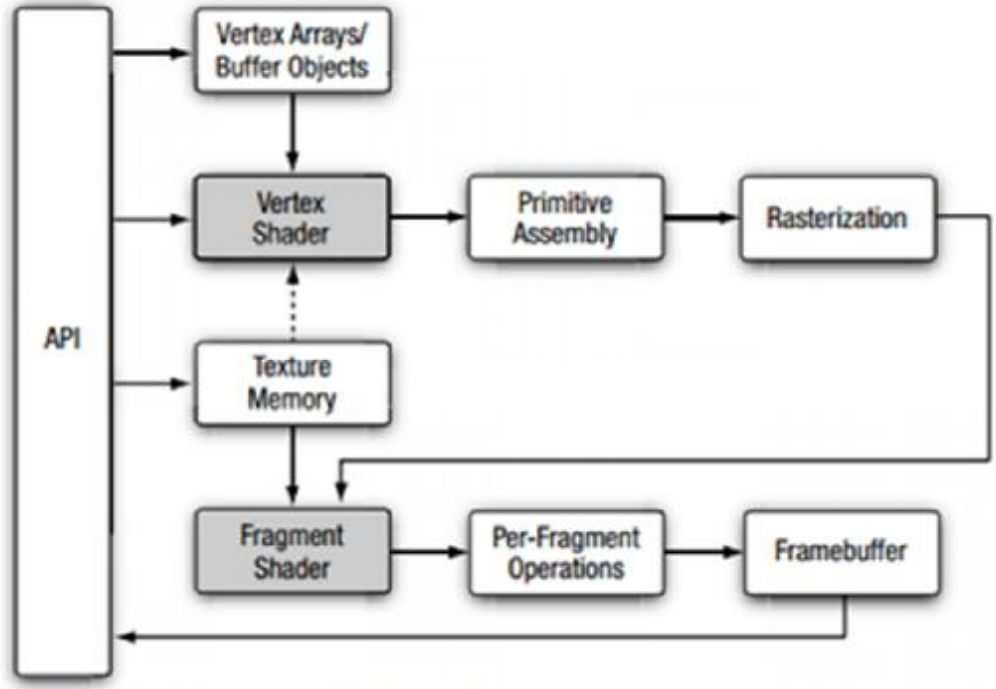


Figure 1-1 OpenGL ES 2.0 Graphics Pipeline

⊕ 渲染管线发展史

- 可以看到这里面灰色的两个Shader就是两个有智商的工人了，API也就变成了给工人下命令（编译、链接着色器代码）就像右边这样

Shaders and Programs

Shader Objects [2.10.1]

uint **CreateShader**(enum *type*);
type: VERTEX_SHADER, FRAGMENT_SHADER

void **ShaderSource**(uint *shader*, sizei *count*, const char *******string*, const int ******length*);

void **CompileShader**(uint *shader*);

void **ReleaseShaderCompiler**(void);

void **DeleteShader**(uint *shader*);

Loading Shader Binaries [2.10.2]

void **ShaderBinary**(sizei *count*, const uint ******shaders*, enum *binaryformat*, const void ******binary*, sizei *length*);

Program Objects [2.10.3]

uint **CreateProgram**(void);

void **AttachShader**(uint *program*, uint *shader*);

void **DetachShader**(uint *program*, uint *shader*);

void **LinkProgram**(uint *program*);

void **UseProgram**(uint *program*);

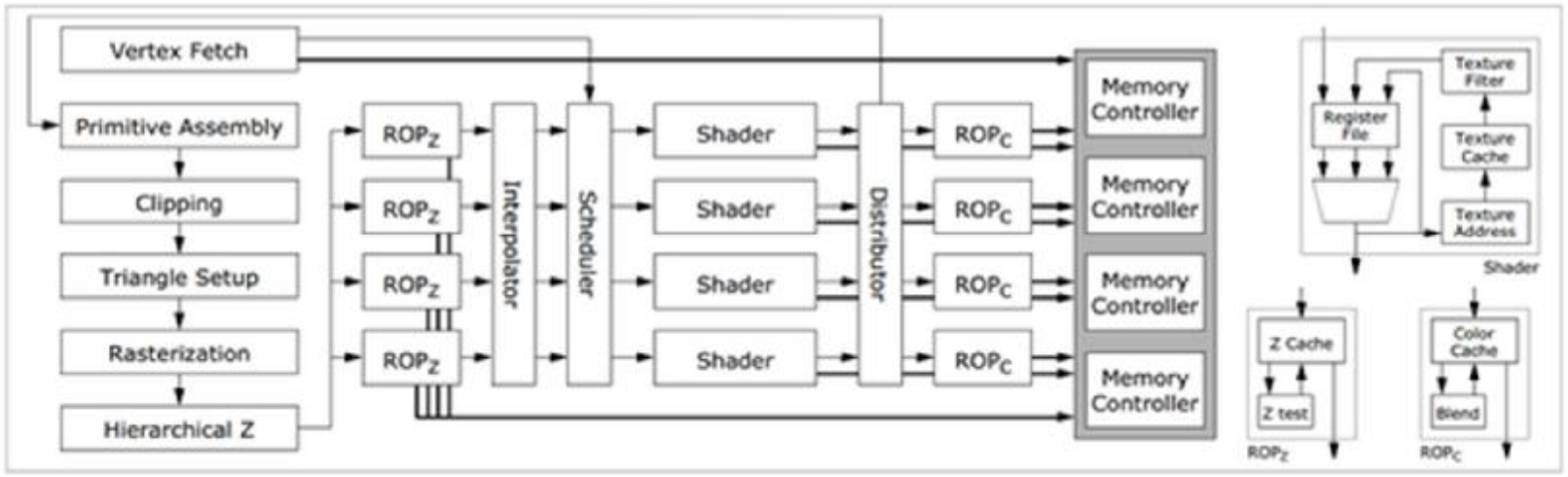
void **DeleteProgram**(uint *program*);

⊕ 渲染管线发展史

- 再到后来大家发现这样分成几个Shader的话，可能会出现任务分配不均的问题，有的时候处理顶点的那个工人忙的要死，然后处理片段的那个在磨洋工，有的时候就反过来。

- 然后大家决定取消工种划分，统一调度劳动力，这个就是Unified Shader（统一渲染架构）了，就像下面这样：

⊕ 渲染管线发展史



不过这个只是硬件的变化，程序员是不可见的，此为后话了。

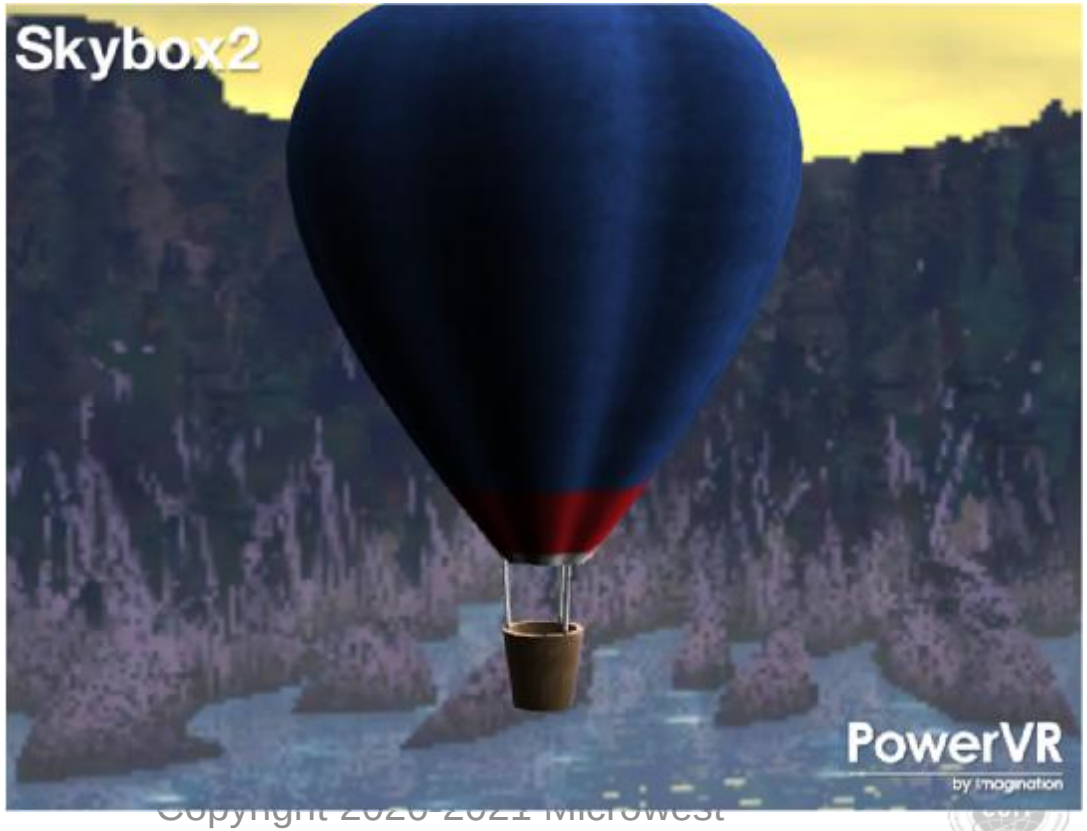
⊕ 渲染管线发展史

- 说了这么多，再补一点可编程和固定管线效果上的区别吧，顺便也可以看出两种管线编程时的不同。以下是PowerVR SDK里面的一个例子

n 这个是固定管线的效果

⊕ 渲染管线发展史

- 这个是可编程的效果



⊕ 渲染管线发展史

- 固定管线时就像右边这样花式设参数

```
glEnable(GL_DEPTH_TEST);

// Enables Smooth Color Shading
glShadeModel(GL_SMOOTH);

// Enable texturing
glEnable(GL_TEXTURE_2D);

// Define front faces
glFrontFace(GL_CW);

// Disable Blending
glDisable(GL_BLEND);

// Enables texture clamping
glTexParameterf( GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE );
glTexParameterf( GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE );

// Set the clear colour
glClearColor(0.5f, 0.5f, 0.5f, 1.0f);

// Reset the model view matrix to position the light
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();

// Setup ambiant light
glEnable(GL_LIGHTING);
float fLightGlobalAmbient[] = {0.4f, 0.4f, 0.4f, 1.0f};
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, fLightGlobalAmbient);

// Setup a directional light source
float fLightPosition[] = {0.7f, 1.0f, -0.2f, 0.0f};
float fLightAmbient[]  = {0.6f, 0.6f, 0.6f, 1.0f};
float fLightDiffuse[]  = {1.0f, 1.0f, 1.0f, 1.0f};
float fLightSpecular[] = {1.0f, 1.0f, 1.0f, 1.0f};

glEnable(GL_LIGHT0);
glLightfv(GL_LIGHT0, GL_POSITION, fLightPosition);
glLightfv(GL_LIGHT0, GL_AMBIENT,  fLightAmbient);
glLightfv(GL_LIGHT0, GL_DIFFUSE,  fLightDiffuse);
glLightfv(GL_LIGHT0, GL_SPECULAR, fLightSpecular);

// Setup the balloon material
float fMatAmbient[] = {0.7f, 0.7f, 0.7f, 1.0f};
float fMatDiffuse[] = {1.0f, 1.0f, 1.0f, 1.0f};
float fMatSpecular[] = {0.0f, 0.0f, 0.0f, 0.0f};

glMaterialfv(GL_FRONT_AND_BACK, GL_AMBIENT,  fMatAmbient);
glMaterialfv(GL_FRONT_AND_BACK, GL_DIFFUSE,  fMatDiffuse);
glMaterialfv(GL_FRONT_AND_BACK, GL_SPECULAR, fMatSpecular);
```

⊕ 渲染管线发展史

- 到了可编程的时代就可以自己写代码来设置各种光照模型了

```
//Effect 1 ////////////////////////////////////////////////////////////////////
//
// This effect displays a textured balloon with some simple lighting.
//

[VERTEXSHADER]
    NAME        balloon_vert1

    [GLSL_CODE]
        attribute highp vec3    myVertex;
        attribute mediump vec3  myNormal;
        attribute mediump vec2  myUV;
        uniform mediump mat4    myMVPMatrix;
        uniform mediump mat3    myModelViewIT;
        uniform mediump vec3    myLightDirection;
        uniform mediump float   fAnim;
        varying mediump float   fDot;
        varying lowp vec2   fTexCoord;

        void main(void)
        {
            gl_Position = myMVPMatrix * vec4(myVertex,1);
            fTexCoord = myUV;
            mediump vec3 fTransNormal = myModelViewIT * myNormal;
            fDot = 0.1 * dot(fTransNormal, myLightDirection) + 0.9;
        }
    [/GLSL_CODE]
[/VERTEXSHADER]

[FRAGMENTSHADER]
    NAME        balloon_frag1

    [GLSL_CODE]
        uniform         sampler2D   sampler2d;
        varying mediump float       fDot;
        varying lowp vec2       fTexCoord;

        void main (void)
        {
            gl_FragColor = texture2D(sampler2d,fTexCoord) * fDot;
        }
    [/GLSL_CODE]
[/FRAGMENTSHADER]
```

⊕ 渲染

```
130  //Effect 2 ////////////////////////////////////////////////////////////////////////
131  //
132  // This effect creates a burning effect that burns the balloon from the basket
133  // up. It does this by calculating whether a pixel is below at least one of
134  // three thresholds (which rise over time). If it is not then the balloon
135  // is textured and lit in the same way as effect 1 otherwise the burn effect is
136  // applied according to the threshold it is below.
137  //
138
139  [VERTEXSHADER]
140      NAME        balloon_vert2
141
142      [GLSL_CODE]
143          attribute vec3  myVertex;
144          attribute vec3  myNormal;
145          attribute vec2  myUV;
146
147          uniform mat4    myMVPMatrix;
148          uniform mat3    myModelViewIT;
149
150          uniform mediump vec3    myLightDirection;
151          uniform highp float fAnim;
152
153          varying float   fDiffuse;
154          varying lowp    vec2 fTexCoord;
155          varying lowp    vec2 fTexCoord2;
156
157          varying highp vec3  fPosition;
158          varying highp float fBurn;
159
160          void main(void)
161          {
162              gl_Position = myMVPMatrix * vec4(myVertex,1);
163
164              fDiffuse = 0.1 * dot(myModelViewIT * myNormal, myLightDirection) + 0.9;
165
166              fPosition =  myVertex * 0.05;
167
168              highp float fOffset = fract(0.999 * fAnim) * 2.1;
169              fBurn = fOffset - 0.53 * (fPosition.y + 0.2);
170              fBurn = clamp(fBurn, 0.0, 1.0);
171
172              fTexCoord = myUV;
173              fTexCoord2 = vec2(fPosition.x + fPosition.z * fPosition.z, fPosition.y + 0.5 * fPosition.z + 0.5 * fPosition.x * fPosition.x);
174          }
175      [/GLSL_CODE]
176  [/VERTEXSHADER]
```

⊕ 渲染管线发展史

- 到了可编
代就可以
代码来设
光照模型

```
[FRAGMENTSHADER]
    NAME        balloon_frag2

    [GLSL_CODE]
        varying highp float fDiffuse;
        varying lowp vec2   fTexCoord;
        varying lowp vec2  fTexCoord2;

        uniform sampler2D sampler2d;
        uniform sampler2D Noise;

        varying highp float fBurn;

        highp vec3 reflect (void)
        {
            return texture2D(sampler2d,fTexCoord).rgb * fDiffuse;
        }

        void main (void)
        {
            highp vec4  noisevec;
            highp vec3  color;
            highp float intensity;

            noisevec = texture2D(Noise, fTexCoord2);

            intensity = 0.6 * (noisevec.x + noisevec.y + noisevec.z + noisevec.w);
            intensity = abs(intensity - 1.0);
            intensity = clamp(intensity, 0.0, 1.0);

            if (intensity < fBurn)
                color = vec3(0.0);
            else if(intensity < 1.5 * fBurn)
                color = vec3(0.1);
            else if(intensity < 1.7 * fBurn)
                color = vec3(1.0, 10.0 * (-intensity + 1.7 * fBurn) ,0.0);
            else
                color = reflect();

            gl_FragColor = vec4(color, 1);
        }
    [/GLSL_CODE]
[/FRAGMENTSHADER]
```

计算机图形学 - 补充资料

结束！

@何晓曦
2020版